

SCAN THIS



JULY 12TH, 2026

DISTRIBUTED JOB SCHEDULER -CODITY ASSIGNMENT

RA2311026010487

[GITHUB](#)

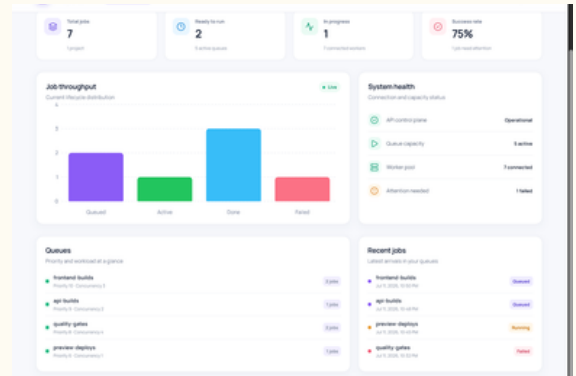
↑
CLICK HERE

Designed and developed a production-inspired distributed job scheduling platform using FastAPI, PostgreSQL, SQLAlchemy (async), React, and Docker, supporting priority queues, worker heartbeats, retry policies, dead-letter queues, JWT authentication, and scheduled job execution.

The Overview UI is implemented in frontend/src/pages/Dashboard.tsx. React fetches jobs, queues, projects, and workers through Axios, derives status metrics in memory, and renders Recharts visualizations. FastAPI exposes authenticated endpoints, while SQLAlchemy queries SQLite or PostgreSQL-backed scheduler data for the dashboard.

Core Technologies

- React 19
- TypeScript
- Vite
- React Router
- Axios
- Recharts - Analytics & dashboards
- Lucide React - Icons

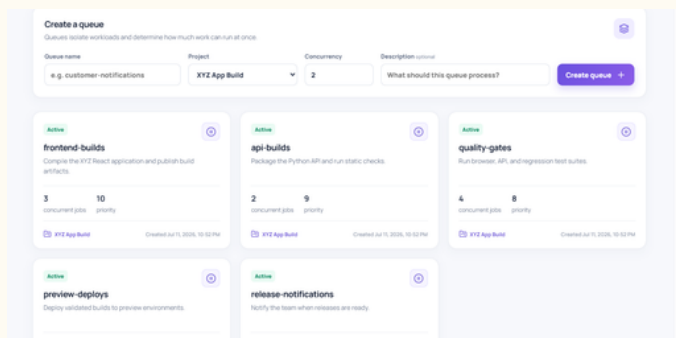


QUEUES

Queues use frontend/src/pages/Queues.tsx to load projects and queues via Axios. The FastAPI queues router verifies project organization membership before storing SQLAlchemy Queue records, retry policies, concurrency limits, priorities, and paused state. The UI posts creations and pause/resume changes, refreshing state.

JOBS

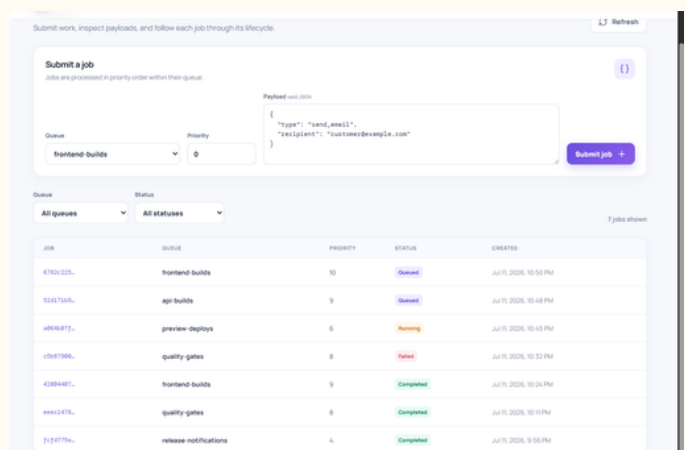
Jobs use frontend/src/pages/Jobs.tsx, which validates JSON payloads and submits authenticated requests. The FastAPI jobs router confirms queue ownership, stores SQLAlchemy Job models, and retrieves filtered results. Worker code conditionally updates status, preventing concurrent workers from claiming the same queued job.



Before creating a job it verifies that the authenticated user actually belongs to the organization owning that queue. This prevents unauthorized access.

Enhanced Experience

Working on this project significantly improved my backend development skills. I learned how to build scalable APIs with FastAPI, design relational databases, implement secure JWT authentication, and develop a distributed job scheduling system with queues, workers, retries, and scheduled execution. It gave me practical experience in building production-style backend applications.



BACK END

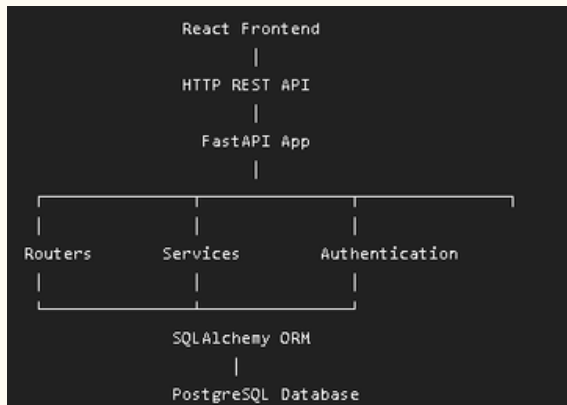
The backend is built as a RESTful API using FastAPI and follows a layered architecture, separating routing, business logic, data models, and database access. Here's how it works.

- Machine Learning
- Natural Language Processing (NLP)
- Computer Vision
- Predictive Analytics

Database Layer

he project uses:

- PostgreSQL
- SQLAlchemy Async ORM
- Async sessions
- database.py creates
- Async Engine
- Session Factory
- Base Model



Create Job

Database
(status = queued)

Worker polls queue

Claims job

Executes

Updates status

Register Worker

Start polling

Look for queued jobs

Claim one

Execute

Save execution result

Schemas , Pydantic models.

They validate incoming API requests and format outgoing responses.

Example:

JobCreate-

contains only the fields required to create a job.

JobResponse-

contains the fields returned to the frontend.

This prevents exposing internal database objects directly.

Body_login_api_auth_login_post	Expanded object
HTTPValidationError	Expanded object
JobCreate	Expanded object
JobResponse	Expanded object
ProjectCreate	Expanded object
ProjectResponse	Expanded object
QueueCreate	Expanded object
QueueResponse	Expanded object
ScheduledJobCreate	Expanded object
Token	Expanded object
UserCreate	Expanded object
UserResponse	Expanded object
ValidationError	Expanded object
WorkerResponse	Expanded object

CORE

auth	Expanded object
projects	Expanded object
queues	Expanded object
jobs	Expanded object
workers	Expanded object
default	Expanded object

AUTH

Login

Verify username/password

Generate JWT Token

Client stores token

Future requests:
Authorization: Bearer <token>

Backend

Python

FastAPI – REST API framework

SQLAlchemy 2.0 (Async ORM) – Database ORM

PostgreSQL – Primary database

AsyncPG – PostgreSQL async driver

Alembic – Database migrations

Pydantic v2 – Data validation & serialization

JWT (python-jose) – Authentication

Passlib (bcrypt) – Password hashing

Croniter – Cron-based job scheduling

Prometheus Client – Metrics & monitoring

Uvicorn – ASGI server